

FULL STACK DEVELOPMENT

FASE 5 | AULA 04 -

APLICANDO TÉCNICAS DE CLEAN CODE EM TESTES NA PRÁTICA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMANIP

O QUE VEM POR AÍ?

Todos os softwares construídos existem para resolver problemas, sejam novos ou aqueles que já existem no mundo real. Estes problemas têm que ser definidos, documentados e analisados para que se entenda como são resolvidos, quais exceções eles tratam e como lidar com as falhas que podem ocorrer durante a resolução do problema.

Os casos de uso são uma ferramenta importante para a análise e documentação dos problemas que o software resolve. Na Arquitetura Limpa, os elementos de software da camada de casos de uso são implementados em uma camada própria e são o “cérebro” do software, fazendo o trabalho de entregar os resultados esperados para cada necessidade.

HANDS ON

Na Arquitetura Limpa, a centralidade dos casos de uso como orquestradores da regra de negócio não pode ser ignorada. Veremos nas videoaulas um papo com vários exemplos, focando em entender como os casos de uso se relacionam com o resto da Arquitetura Limpa. Gostou?! Assista os vídeos para entender ainda mais!

Para essa aula, temos um link para você poder treinar o conteúdo ensinado. Acesse:

[Base de dados da aula 04.](#)

EXEMPLO

SAIBA MAIS

Use cases como ferramenta

A ideia de documentar as necessidades de uso do software de acordo com os “casos de uso” não é nova. Há muito tempo, as principais metodologias de construção de software, e até mesmo de gestão de processos de software, consideram a documentação dos casos de uso como uma das principais ferramentas para a organização das necessidades dos(as) clientes, usuários(as) e interessados(as) (stakeholders) do software. Existem até mesmo modelos de criação de casos de uso e livros que falam sobre o assunto.

Um caso de uso é a documentação de um processo de negócio, considerando os dados necessários para fazer o processo de negócio acontecer, quais passos devem ser seguidos para finalizar o processo, quais são as exceções que podem ocorrer e como devem ser tratadas, e qual o resultado desse processo de negócio.

Portanto, pensar nos casos de uso como base para o sucesso do software faz sentido quando o maior interesse é fazer com que o software resolva problemas. E quando aplicamos o pensamento dos casos de uso para processos de metodologias ágeis, utilizando os casos de uso implementamos apenas o que é necessário, não o que é desejado. Usualmente, os cards de implementação em metodologias ágeis são baseados em casos de uso por este motivo.

Use cases como camada de arquitetura

No desenho da figura 1 – “Arquitetura Limpa”, vemos uma camada de casos de uso definida de forma isolada entre a camada de entidades e a camada de controllers:

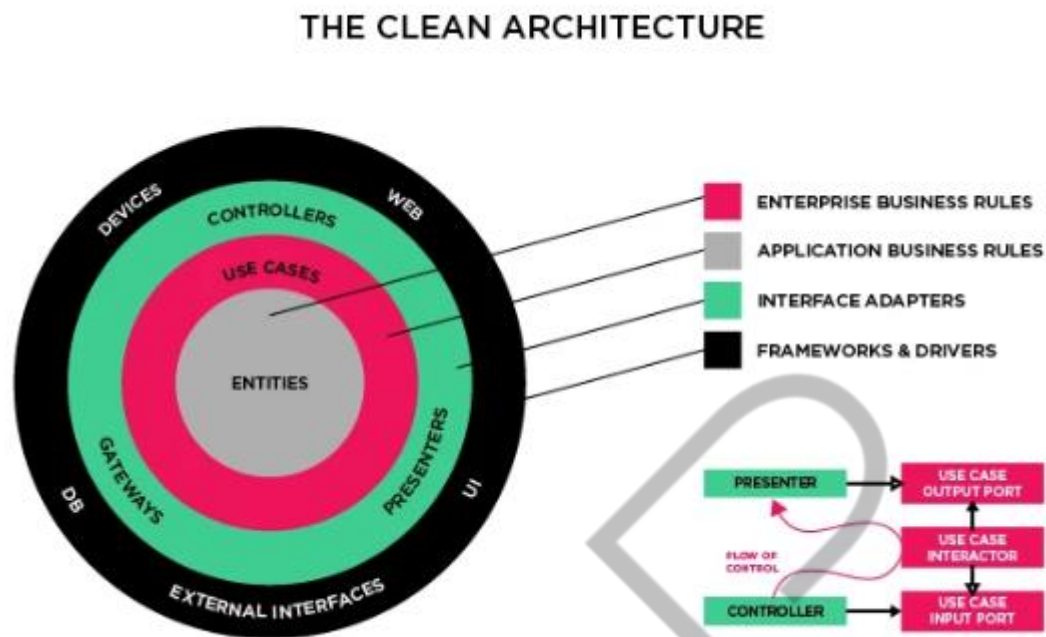


Figura 1 – Arquitetura Limpa

Fonte: [https://www.freecodecamp.org/portuguese/news/uma-breve-introducao-a-arquitetura-limpa/\(2023\)](https://www.freecodecamp.org/portuguese/news/uma-breve-introducao-a-arquitetura-limpa/(2023)), adaptado por FIAP (2023)

Para a Arquitetura Limpa, os casos de uso são a parte do software que entrega as funcionalidades para os elementos externos e centraliza a implementação das regras de negócio. Isso faz sentido quando pensamos que as duas camadas internas são as camadas dedicadas a definir “o que” o software faz, enquanto a camada de casos de uso define “como” o software faz.

É nessa camada que vamos implementar, um de cada vez, todos os casos de uso necessários para a entrega do software. Esse isolamento é importante porque nos permite:

- Isolar cada regra de negócio em seu código único, testável e utilizável, seja por outros casos de uso ou pelas camadas mais externas.
- Implementar novas regras de negócio sem o risco de causar problemas em regras já existentes.
- Segregar o funcionamento do núcleo do software, que cuida das responsabilidades relacionadas ao que o software faz e da implementação das camadas mais técnicas, que cuidam da comunicação com o mundo exterior.

Para isso, vamos usar as entidades que criamos na camada interna e criar o código que implementa o que cada caso de uso define, de forma isolada, seguindo as regras do SOLID. Por exemplo:

- Cada implementação nessa camada deve resolver apenas um caso de uso. Não é aceitável criar um caso de uso que faça mais de uma coisa. E quando falamos fazer, significa entregar o resultado esperado de acordo como definido. Um caso de uso não pode, por exemplo, criar um Estudante a partir de uma Pessoa e matricular este Estudante na matéria; neste caso, ele deve emitir um erro informando que o dado (Pessoa) recebido é inválido. Em casos complexos, podemos dividir um caso de uso em casos de uso menores, e com isso manter o isolamento de responsabilidade. Se precisar de outros casos de uso, deve-se implementar a chamada a este de forma explícita.
- Os casos de uso podem ser agrupados em classes e módulos/bibliotecas/pacotes. Esse agrupamento será feito de acordo com alguma regra de organização coerente com a organização do software, e não de uma forma arbitrária. Por exemplo, podemos implementar todos os casos de uso relacionados ao processo de matrícula de alunos em um único pacote ou em uma classe. No caso do uso de uma classe, precisamos implementar isso como um grupo de métodos estáticos, uma vez que os casos de uso devem funcionar de forma independente, sem dividir estado.
- Os casos de uso devem usar as entidades como a abstração dos atores em cada caso de uso, e nunca usar diretamente os dados recebidos do código que o instanciou. Criando as entidades já é possível validar os dados, e posteriormente usar cada uma das instâncias de entidade dentro do código.

Usando os Casos de uso

Um caso de uso é composto de três partes: os dados que recebe, o algoritmo interno que implementa a funcionalidade, e o retorno do caso de uso.

A primeira parte é composta por um ou mais dados recebidos no momento de instanciar a regra de negócio. Estes dados usualmente vêm definidos como um tipo de dados particular, que pode ser criado como uma interface, para padronizar o acesso a este caso de uso, ou pode receber diretamente uma instância de uma entidade. Se vamos criar um Caso de uso que faz o cadastro de uma pessoa em nossa faculdade, podemos ter uma assinatura de caso de uso neste formato:

```
class EstudanteUseCases {  
    public static registrarEstudante(p: Pessoa) {  
        // ...  
    }  
}
```

Figura 2 – Caso de uso 1
Fonte: Elaborado pelo autor (2023)

Para este caso de uso, estamos definindo como registrar um estudante, que é o ato de criar um objeto Estudante a partir de um objeto Pessoa já existente. Vamos melhorar esta assinatura incluindo um serviço que usaremos:

```
class EstudanteUseCases {  
    public static registrarEstudante(p: Pessoa, estudanteRepositorio: EstudanteRepositorio): Estudante {  
        /* caso de uso: registrar uma pessoa como estudante */  
        if (EstudanteUseCases.verificarPessoaEstudante(p, estudanteRepositorio) != null) {  
            throw new Error("Pessoa ja e estudante");  
        }  
        return new Estudante(p.nome, p.identificacao, 'nova_matricula');  
    }  
  
    public static verificarPessoaEstudante(p: Pessoa, estudanteRepositorio: EstudanteRepositorio) {  
        return estudanteRepositorio.buscarPorPessoa(p.identificacao);  
    }  
}
```

Figura 3 – Caso de uso 2
Fonte: Elaborado pelo autor (2023)

Para este caso de uso, melhoramos o processo: agora validamos se esta Pessoa já é Estudante verificando no repositório de dados. Mas o objetivo aqui é apenas conseguir criar Estudante, não registrar no repositório de dados, uma vez que este não é o objetivo do caso de uso. O objetivo do caso de uso é apenas criar um objeto e validar se está de acordo com o esperado. Possivelmente, no processo de chamada desse caso de uso, quem o chamou espera os dados para registrar no banco de dados, usando diretamente o Repositório por meio de um Adapter.

O caso de uso recebeu uma **instância** de um **repositório de dados** para usar em seu processo. Isso é a aplicação direta da regra de Inversão de Dependência: podemos criar um repositório “falso”, de teste, para verificar se a regra de negócio funciona como esperado. Funcionando, temos a garantia de que qualquer erro que acontecer está dentro do repositório, não em nosso código.

O caso de uso também usa outro caso de uso, para verificar se Pessoa já é Estudante, porque o objetivo do caso de uso é apenas registrar Estudante, ou seja, retornar um objeto Estudante que seja válido. Nesse momento, o caso de uso é simples, mas está totalmente isolado e faz apenas uma coisa. Mesmo os casos de uso mais complexos seguem essa regra, de terceirizar seu trabalho para outros casos de uso caso seja necessário.

O retorno do caso de uso, nesse momento, é um objeto do tipo Estudante, que está definido na camada de entidade. Como a classe usuária é “externa” à camada de entidades, isso não viola a regra das camadas. Em algumas implementações, podemos retornar um objeto de dados (DAO) ao invés de retornar uma instância da entidade. Isso depende muito da implementação feita e de como a implementação deve ser feita. De qualquer forma, esse objeto de Dados serve mais como uma interface de comunicação (como uma porta, por exemplo) do que como uma imposição da Arquitetura Limpa.

Use cases para teste do software

Neste momento, depois de ver apenas um caso de uso implementado, já podemos testar este outro caso de uso; podemos inicializar manualmente uma entidade, criar uma implementação “mock” do repositório para informar a este caso de uso, e validar se funciona corretamente.

Antes de falarmos mais sobre isso, é importante destacarmos que uma implementação “mock” é um objeto que se comporta exatamente como a interface pede, e retorna os dados necessários para fazer com que o código do teste funcione. O nome “mock” não tem correspondente em português, sendo que algumas transliterações já usaram o termo “fantoche”, que parece fazer sentido: é um objeto que “se comporta como” um objeto real, mas que foi manipulado para um contexto, e para um propósito específico.

Mas então, qual a vantagem disso? Simples, desta forma nós testamos as regras de negócio e a funcionalidade da aplicação de acordo com as funcionalidades desejadas, antes de conectar às interfaces de comunicação externa. Isso garante que o software faz o que precisa fazer, independentemente de como será usado.

Posteriormente, vamos ver como conectar estas camadas de regra de negócios ao “mundo real”, mas nesse momento conseguimos separar, de forma clara, o núcleo da aplicação de tudo o que a cerca. Usando nosso exemplo, podemos garantir que é possível sempre tentar criar um Estudante a partir de uma Pessoa, não importa se o cliente pede isso a partir de uma API, de um acesso via GUI ou qualquer outra interface que ainda não conhecemos.

O QUE VOCÊ VIU NESTA AULA?

Casos de uso são uma ideia importante na prática da construção de software, usados há muitas décadas e por diferentes metodologias de software como uma ferramenta de captura, entendimento e documentação das necessidades de negócio que devem ser implementadas em software.

Partindo dessa ideia, a Arquitetura Limpa define uma camada de casos de uso para centralizar a implementação dessas regras de forma independente de seu uso. O conceito é radical ao propor que cada caso de uso que seja implementado como uma unidade seja responsável apenas pela funcionalidade esperada, e não seja dependente de nenhum mecanismo externo.

Os casos de uso recebem os dados e dependências de que precisam para funcionar via injeção de dependência e interfaces. Com isso, as regras são relativas apenas ao negócio, e assim conseguimos isolar, com os casos de uso e as entidades, “o que” o software faz.

O que achou desta aula? Conte-nos no Discord!

REFERÊNCIAS

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. [s.l.]: Addison-Wesley Professional, 2003.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, R. **Clean Code: A Handbook of Agile Software Craftsmanship**. [s.l.]: Pearson, 2008.

USE case. Disponível em: <<https://martinfowler.com/bliki/UseCase.html>>. Acesso em: 01 jun. 2023.

PALAVRAS-CHAVE

Casos de Uso. Testes de software. Camadas.

EXEMPLO

POS TECH